Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit ✓ System __ **Test Date: 03/29/2025**

Test Case ID#: 001 **Name(s) of Testers: Elsdon Chang**

Test Description: Testing the promptForInput() in the Election class. This test ensures that promptForInput() correctly parses input and handles errors.

Test file: Project1/testing/ElectionTest.java method/functions being used: promptForInput(), processBallotFile()

Automated: yes ✓ no

Results: Pass ✓ Fail_____

Preconditions for Test: Election instance is created.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Simulate valid input for plurality election (p), valid CSV path, and seats.	p Project1/testing/plurality/0tie1seat.csv 1	Algorithm set to “plurality”	Algorithm was set to “plurality”	Tests plurality input handling.
2	Simulate valid input for stv election (s), valid CSV path, audit filename, shuffle and seats.	s Project1/testing/stv/elimination.csv auditFile 1 1	Algorithm set to “stv”	Algorithm was set to “stv”	Tests stv input handling.
3	Simulate invalid election type (t) to trigger an exception.	t Project1/testing/stv/elimination.csv auditFile 1 1	Exception thrown for invalid election type.	An exception was thrown for invalid election type.	Tests error handling for invalid election type input.
4	Simulate empty election type input to trigger an exception.	Project1/testing/stv/elimination.csv auditFile 1 1	Exception thrown for invalid election type.	An exception was thrown for invalid election type.	Tests error handling for invalid election type input.
5	Simulate invalid CSV path	p invalid.csv 1	Exception thrown for invalid ballot file	An exception was thrown for invalid ballot file path.	Tests error handling for invalid ballot file input.

Post condition(s) for Test: The election’s input object contains all of the information if the input was valid, otherwise the system will display an error message and the program will terminate.

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit ✓ System __

Test Date: 03/29/2025

Test Case ID#: 002

Name(s) of Testers: Cindy Su

Test Description: This tests BallotFileReader's readCandidates function.

Test file: Project1/testing/BallotFileReaderTest.java
methods being used: readCandidates()

Automated: yes ✓ no __

Results: Pass ✓ Fail

Preconditions for Test: a valid ballot file must be present

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	simulate reading candidates from a ballot file	Project1/testing/plurality/2tie1seat.csv	{"A", "B", "C"}	{"A", "B", "C"}	This test validates that the correct candidates are read from the ballot file
2					
3					
4					

Post condition(s) for Test: Candidates that were read from the ballot file are returned as a String[].

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit ✓ System __

Test Date: 03/29/2025

Test Case ID#: 003

Name(s) of Testers: Cindy Su

Test Description: This tests BallotFileReader's readBallots function.

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes ✓ no ___

Results: Pass ✓ Fail ___

Preconditions for Test: a valid ballot file must be present

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	simulate reading plurality ballots from a file	Project1/testing/plurality/2tie1seat.csv	Expected ballots match the ones that are read from the file	Ballots read from the file matched the expected ballots.	Validates that ballots read from a plurality election csv file are correctly processed.
2	simulate reading stv ballots from a file	Project1/testing/stv/elimination.csv	Expected ballots match the ones that are read from the file	Ballots read from the file matched the expected ballots	Validates that ballots read from a stv election csv file are correctly processed.
3					
4					

Post condition(s) for Test: Ballots that were read from the csv file are returned as a List<Ballot>.

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit ✓ System __

Test Date: 03/29/2025

Test Case ID#: 004

Name(s) of Testers: Kongmeng Thao

Test Description: This test Plurality Voting Algorithm

Indicate where are you storing the tests (what file) and the name of the method/functions being used.

Automated: yes ✓ no __

Results: Pass ✓ Fail

Preconditions for Test: the valid input files in Plurality must be present

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Simulate running Election and compare output for first csv test input	Project1/testing/plurality/0tie1seat.csv	winner:{A} Loser: {B , C}	winner:{A} loser: {B , C}	
2	Same thing but different csv input	Project1/testing/plurality/2tie1seat.csv	One winner A or B Two losers C and A or B	one winner two losers (C always lose)	The results is random
3	Same thing but different csv input	Project1/testing/plurality/2tie2seat1.csv	Two winners A and B One losers C	winner: {A, B} loser {C}	
4	Same thing but different csv input	Project1/testing/plurality/2tie2seat2.csv	Two winners A and B or C One losers C or B	two winner (A always win) one loser	The results is random
5	Same thing but different csv input	Project1/testing/plurality/3tie1seat.csv	One winners A or B or C Two losers C or B or A	one winner two loser	The results is random either can win
6	Same thing but different csv input	Project1/testing/plurality/3tie2seat1.csv	Two winners A or B or C One losers C or B or A	two winner one loser	The results is random either can win or lose
7	Same thing but different csv input	Project1/testing/plurality/3tie2seat2.csv	Two winners A and B or C or D Two losers C or B or D	two winner (A always win) two loser	The results is random

Post condition(s) for Test: All input CSV files is valid and the PluralityAlgorithm is able to determined winner based of the CSV files

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit ✓ System __ **Test Date: 03/29/2025**

Test Case ID#: 005 **Name(s) of Testers: Ly Xiong**

Test Description: This test STV

File location: Project1/testing/stv/STVAlgorithmTesting.java

Automated: yes ✓ no __

Results: Pass ✓ Fail _____

Preconditions for Test: valid input for STV is require

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1					
2	Test Droop Quota: calculate droop quota using default seats	100 ballots with 2 seats	34	34	(100/(2+1) + 1) = 34
3	Test Droop Quota: calculate droop quota reading seat from input	3 ballots with 1 seats	2	2	(3/(1+1) + 1) = 2
4	Test ballot shuffle read from input	3 ballots, shuffle == false	The order of ballots should be the same	Same order	
5	Test ballot shuffle using default true	3 ballots, shuffle == true	The order of ballots should be different	Different Order	
6	Test redistribute ballots to candidates:	5 ballots, 3 candidate	A is the winner, C and B are the losers C should be the first to loser follow by B	Winner: A Losers: [C,B]	

Post condition(s) for Test: Calculate the Droop Quota, shuffle ballot (depends on user input) and determine winners and losers

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit __ System ✓ **Test Date: 03/29/2025**

Test Case ID#: 006 **Name(s) of Testers: Elsdon Chang**

Test Description: This system test validates that a plurality election is properly executed through manual inputs from the terminal.

Test Files: Project1/testing/plurality/*.csv

Automated: yes no ✓

Results: Pass ✓ Fail

Preconditions for Test: Valid plurality ballot csv files must be present in Project1/testing/plurality/ folder.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Simulate a plurality election with 0 ties and 1 seat.	testing/plurality/0tie1seat.csv	A wins	A wins	
2	Simulate a plurality election with 2 ties for 1 seat.	testing/plurality/2tie1seat.csv	Winner is A or B and C always loses.	Winner is A or B and C always loses.	
3	Simulate a plurality election with 2 ties for 2 seats for seat 1.	testing/plurality/2tie2seat1.csv	A and B win, C loses	A and B win, C loses	
4	Simulate a plurality election with 2 ties for 2 seats for seat 2.	testing/plurality/2tie2seat2.csv	A always wins, B or C randomly wins seat 2.	A always wins, B or C randomly wins seat 2.	
5	Simulate a plurality election with 3 ties for 1 seat.	testing/plurality/3tie1seat.csv	A or B or C wins randomly.	A or B or C wins randomly.	
6	Simulate a plurality election with 3 ties for 2 seats for seat 1.	testing/plurality/3tie2seat1.csv	Random 2 winners	Random 2 winners	
7	Simulate a plurality election with 3 ties for 2 seats for seat 2.	testing/plurality/3tie2seat2.csv	A always wins and B,C,D randomly wins seat 2.	A always wins and B,C,D randomly wins seat 2.	

Post condition(s) for Test: The results of the election are printed to the screen.

Project Name: Project 1: Voting System **Team#2**

Test Stage: Unit __ System ✓ **Test Date: 03/29/2025**

Test Case ID#: 007 **Name(s) of Testers: Elsdon Chang**

Test Description: This system test validates that a stv election is properly executed through manual inputs from the terminal.

Test Files: Project1/testing/stv/*.csv

Automated: yes___ no ✓

Results: Pass ✓ Fail

Preconditions for Test: Valid plurality ballot csv files must be present in Project1/testing/stv/ folder.

Step #	Test Step Description	Test Data	Expected Result	Actual Result	Notes
1	Simulate a stv election with an elimination where none of the candidates reach droop quota in round 1	testing/stv/elimination.csv	A wins	A wins	
2	Simulate a stv election with a tie for elimination where none of the candidates reach droop quota in round 1	testing/stv/eliminationTie.csv	Winner is A after ballot redistribution.	Winner is A after ballot redistribution.	
3	Simulate a stv election where a candidate reaches droop quota on round 1	testing/stv/quotaMet.csv	A wins	A wins	
4	Simulate a stv election where there is a surplus of votes for a candidate and the ballot needs to be redistributed to the next preference	testing/stv/redistribute.csv	A and B wins	A and B wins	

Post condition(s) for Test: The results of the election are written to the audit file.